МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Державний університет «Житомирська політехніка»
Кафедра комп’ютерної інженерії

ЗВІТ
з лабораторної роботи 3

з дисципліни «Програмування»

Одновимірні та багатовимірні масиви,
обробка символьних рядків у мові С
та робота з гілками в Git

Варіант 3

Виконав студент групи КІ-26-1
Волохов Дмитрій

Житомир 2026

1 Мета роботи

Набути навичок обробки одновимірних масивів і двовимірних матриць у мові C, ручного перетворення символьних рядків та роботи з окремою функціональною гілкою Git. Перевірити результати виконання програм і підготувати пояснювальну записку у форматі PDF.

2 Середовище виконання

Роботу виконано у Windows із використанням MSYS2 UCRT64, компілятора GCC, редактора Visual Studio Code та системи контролю версій Git. Сирцеві файли task1.c, task2.c і task3.c розміщено в каталозі programming-sem1/lab03. Для лабораторної використано гілку lab03.

Компіляція кожної програми виконується з прапорцями -Wall і -Wextra. Вони вмикають додаткові попередження компілятора. Результати запусків наведено на власних скриншотах тестування.

3 Індивідуальні завдання варіанта 3

Завдання 1. Знайти номер мінімального елемента та суму елементів масиву, розташованих між першим і останнім додатними. Масив фіксованого розміру заповнити псевдовипадковими цілими числами з діапазону від −50 до 50.

Завдання 2. Знайти рядок матриці з максимальною сумою елементів та інвертувати знаки всіх його елементів на протилежні. Виконувати обхід матриці за рядками, а значення виводити з фіксованою шириною полів.

Завдання 3. Перетворити всі малі латинські літери введеного рядка на великі шляхом прямого зсуву кодів ASCII. Для введення використати fgets, а перетворення виконати вручну до нуль-термінатора без готових функцій зміни рядків.

Основні принципи реалізації

Початкове значення мінімуму береться з a[0]. Це забезпечує правильний пошук і для масиву, усі елементи якого додатні або від’ємні. Номер елемента в результаті відлічується від 1, а індекс у програмі — від 0.

У C елементи двовимірного масиву зберігаються послідовно за рядками — Row-Major Order. Тому зовнішній цикл перебирає рядки, а внутрішній стовпці. Символьний рядок завершується байтом \0; буфер на 255 однобайтових символів містить 256 байтів разом із термінатором.

4 Програма обробки одновимірного масиву

Файл task1.c. Розмір масиву — 15 елементів. Якщо мінімум повторюється, програма обирає його перше входження.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 15

int main(void)
{
    int a[SIZE];
    srand((unsigned int)time(NULL));

    printf("Initial array:\n");
    for (int i = 0; i < SIZE; ++i) {
        a[i] = rand() % 101 - 50;
        printf("%5d", a[i]);
    }
    printf("\n");

    int minimum = a[0];
    int min_index = 0;
    int first_positive = -1;
    int last_positive = -1;

    for (int i = 0; i < SIZE; ++i) {
        if (a[i] < minimum) {
            minimum = a[i];
            min_index = i;
        }
        if (a[i] > 0) {
            if (first_positive == -1) {
                first_positive = i;
            }
            last_positive = i;
        }
    }

    printf("Minimum: %d\n", minimum);
    printf("Position (from 1): %d\n", min_index + 1);
    printf("Index (from 0): %d\n", min_index);

    if (first_positive == -1 || first_positive == last_positive) {
        printf("Fewer than two positive elements: no interval.\n");
    } else {
        int sum = 0;
        for (int i = first_positive + 1; i < last_positive; ++i) {
            sum += a[i];
        }
        printf("First positive position: %d\n", first_positive + 1);
        printf("Last positive position: %d\n", last_positive + 1);
        printf("Sum between them: %d\n", sum);
    }

    printf("Array after processing:\n");
    for (int i = 0; i < SIZE; ++i) {
        printf("%5d", a[i]);
    }
    printf("\n");
    return 0;
}
```

5 Перевірка першого завдання

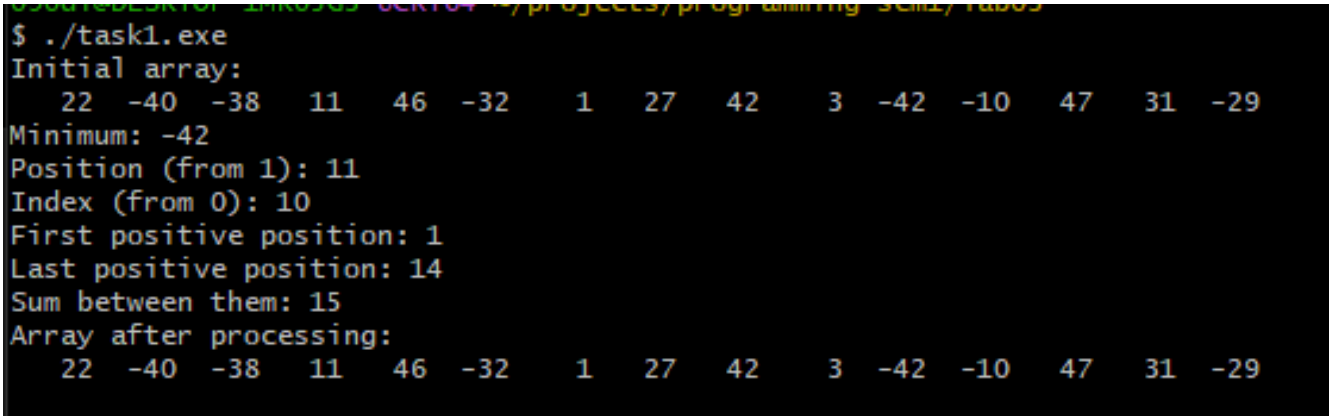


Рисунок 1 — Результат запуску програми task1.c

У зафіксованому запуску отримано такий масив:

```
22  -40  -38  11  46  -32  1  27  42  3  -42  -10  47  31  -29
```

Показник	Значення
Мінімальний елемент	−42
Номер мінімального елемента	11
Індекс мінімального елемента	10
Перше додатне число	22, позиція 1
Останнє додатне число	31, позиція 14
Сума між крайніми додатними	15

Для перевірки суми враховано елементи з позицій 2–13. Крайні додатні числа 22 і 31 до суми не входять:

−40 − 38 + 11 + 46 − 32 + 1 + 27 + 42 + 3 − 42 − 10 + 47 = 15.

Розраховані вручну значення збігаються з результатами програми. Масив після обробки залишається незмінним, оскільки індивідуальне завдання передбачає пошук та обчислення суми.

Програма окремо обробляє випадок, коли додатних елементів менше двох: повідомляє про відсутність інтервалу. Якщо два крайні додатні елементи сусідні, сума порожнього інтервалу дорівнює нулю. Часова складність алгоритму — O(N).

Команди компіляції та запуску:

```
gcc -Wall -Wextra task1.c -o task1.exe
./task1.exe
```

6 Програма обробки двовимірної матриці

Файл task2.c. Використано прямокутну матрицю розміром 3 × 5. Суми обчислюються за початковими значеннями до зміни знаків.

```
#include <stdio.h>

#define ROWS 3
#define COLS 5

void print_matrix(int a[ROWS][COLS])
{
    for (int i = 0; i < ROWS; ++i) {
        for (int j = 0; j < COLS; ++j) {
            printf("%6d", a[i][j]);
        }
        printf("\n");
    }
}

int main(void)
{
    int a[ROWS][COLS] = {
        {4, -2, 7, 1, -5},
        {8, 3, -1, 6, 4},
        {-6, 2, 5, -3, 1}
    };
    int row_sums[ROWS] = {0};

    printf("Initial matrix:\n");
    print_matrix(a);

    for (int i = 0; i < ROWS; ++i) {
        for (int j = 0; j < COLS; ++j) {
            row_sums[i] += a[i][j];
        }
        printf("Row %d sum: %d\n", i + 1, row_sums[i]);
    }

    int max_row = 0;
    for (int i = 1; i < ROWS; ++i) {
        if (row_sums[i] > row_sums[max_row]) {
            max_row = i;
        }
    }

    printf("Selected row: %d; sum: %d\n",
           max_row + 1, row_sums[max_row]);

    for (int j = 0; j < COLS; ++j) {
        a[max_row][j] = -a[max_row][j];
    }

    printf("Matrix after processing:\n");
    print_matrix(a);
    return 0;
}
```

7 Перевірка другого завдання

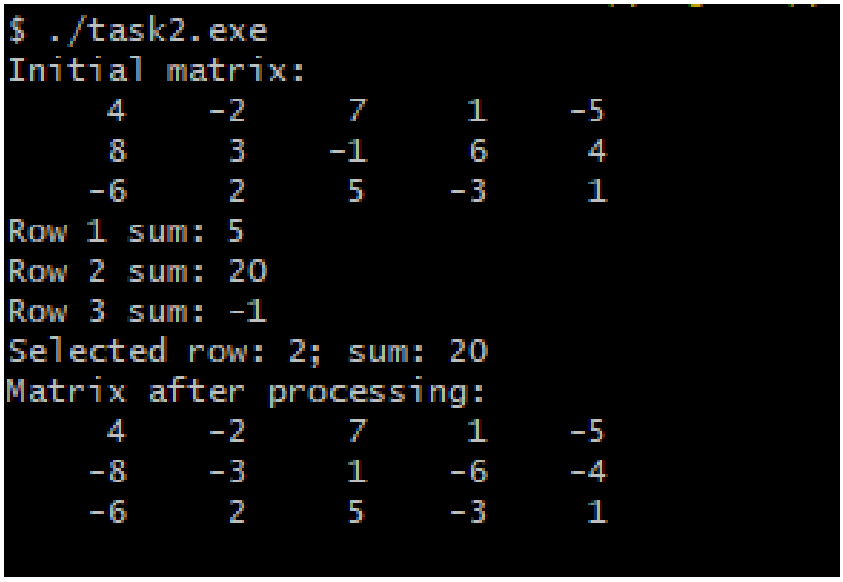


Рисунок 2 — Матриця до обробки та після зміни знаків

Рядок	Перевірка суми	Сума
1	4 − 2 + 7 + 1 − 5	5
2	8 + 3 − 1 + 6 + 4	20
3	−6 + 2 + 5 − 3 + 1	−1

Найбільша сума дорівнює 20 і належить другому рядку. Після інвертування знаків цього рядка отримано:

```
До обробки:       8   3  -1   6   4
Після обробки:   -8  -3   1  -6  -4
```

Перший і третій рядки залишаються незмінними. Сума зміненого другого рядка дорівнює −20. Це підтверджує, що знак змінено для кожного його елемента, зокрема −1 перетворено на 1.

Масив row_sums зберігає початкові суми рядків. Пошук починається з нульового рядка, тому алгоритм працює і тоді, коли всі суми від’ємні. За рівності найбільших сум обирається перший відповідний рядок.

Виведення та підрахунок сум здійснюються вкладеними циклами: спочатку індекс рядка i, потім стовпця j. Перетворення вибраного рядка також послідовно обходить його елементи. Загальна часова складність — O(R × C).

Команди компіляції та запуску:

```
gcc -Wall -Wextra task2.c -o task2.exe
./task2.exe
```

8 Програма обробки символьного рядка

Файл task3.c. Буфер містить 256 байтів. Кожну літеру від a до z замінено символом із кодом, меншим на 32 — різницю між кодами a та A. Інші символи збережено.

```
#include <stdio.h>

#define MAX_TEXT 255

int main(void)
{
    char str[MAX_TEXT + 1];

    printf("Enter text (up to 255 characters):\n");
    if (fgets(str, sizeof(str), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }

    for (int i = 0; str[i] != '\0'; ++i) {
        if (str[i] == '\n') {
            str[i] = '\0';
            break;
        }
        if (str[i] >= 'a' && str[i] <= 'z') {
            str[i] = (char)(str[i] - ('a' - 'A'));
        }
    }

    printf("Result: %s\n", str);
    return 0;
}
```

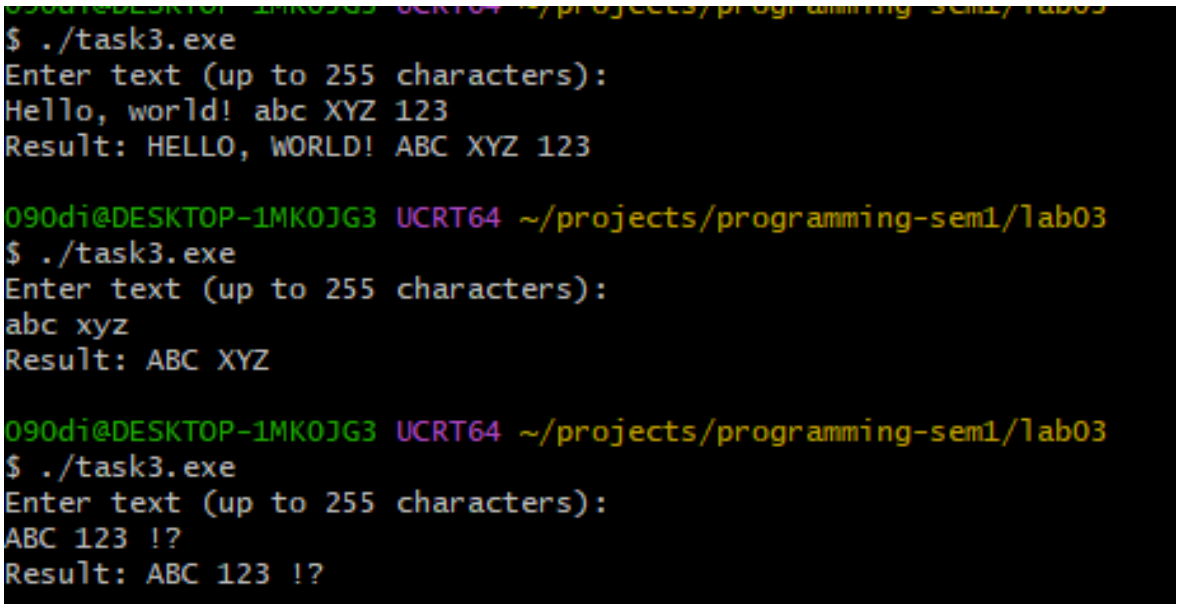


Рисунок 3 — Три перевірки перетворення латинських літер

Отримано HELLO, WORLD! ABC XYZ 123, ABC XYZ та ABC 123 !?. Малі літери стали великими; цифри, пробіли, розділові знаки й уже великі літери збережено. Перенесення рядка від fgets замінено на \0. Сканування має складність O(L), де L — довжина введеного тексту.

9 Робота з гілкою Git

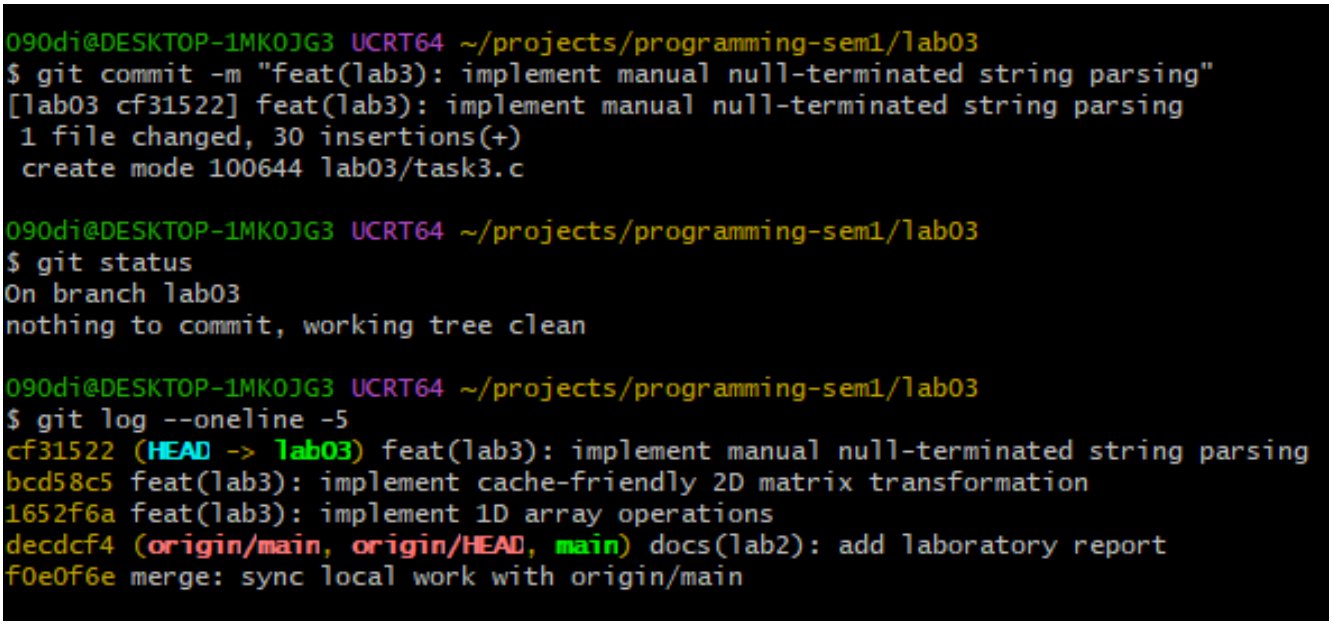


Рисунок 4 — Коміти лабораторної та стан робочого дерева

Коміт	Зміст
1652f6a	Реалізація обробки одновимірного масиву
bcd58c5	Реалізація перетворення двовимірної матриці
cf31522	Реалізація ручної обробки символьного рядка

На скриншоті HEAD указує на гілку lab03 і коміт cf31522. Команда git status підтверджує відсутність незафіксованих змін. На цьому етапі гілка main ще не містить комітів лабораторної роботи 3.

Після збереження Report_Lab03.pdf у каталозі lab03 завершення подання виконується такими командами з цього каталогу:

```
git add Report_Lab03.pdf
git commit -m "docs(lab3): add mandatory engineering PDF report"
cd ..
git switch main
git merge --no-ff lab03 -m "merge: integrate lab03"
git push -u origin main
git push -u origin lab03
```

10 Висновки

Реалізовано та перевірено три програми мовою C для варіанта 3. Для масиву правильно знайдено мінімум −42 на позиції 11 та суму 15 між крайніми додатними. У матриці визначено другий рядок із сумою 20 і змінено знаки його елементів. Перетворення латинських літер виконано вручну з дотриманням меж буфера та збереженням нуль-термінатора. Зміни зафіксовано окремими комітами в гілці lab03.

Джерело завдань: методичні вказівки «Лабораторне заняття №3», розділи 3–5, таблиці 1–3, варіант 3.